

WEEK-07

Implement 0/1 Knapsack problem using dynamic programming.

```
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int knapsack(int W, int weights[], int values[], int n) {
    int dp[n+1][W+1];

    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {

            if (i == 0 || w == 0)
                dp[i][w] = 0;

            else if (weights[i-1] <= w)
                dp[i][w] = max(dp[i-1][w],
                               dp[i-1][w - weights[i-1]] + values[i-1]);

            else
                dp[i][w] = dp[i-1][w];
        }
    }
    return dp[n][W];
}

int main() {
    int n, W;
    printf("Enter number of items: ");
    scanf("%d", &n);

    int weights[n], values[n];

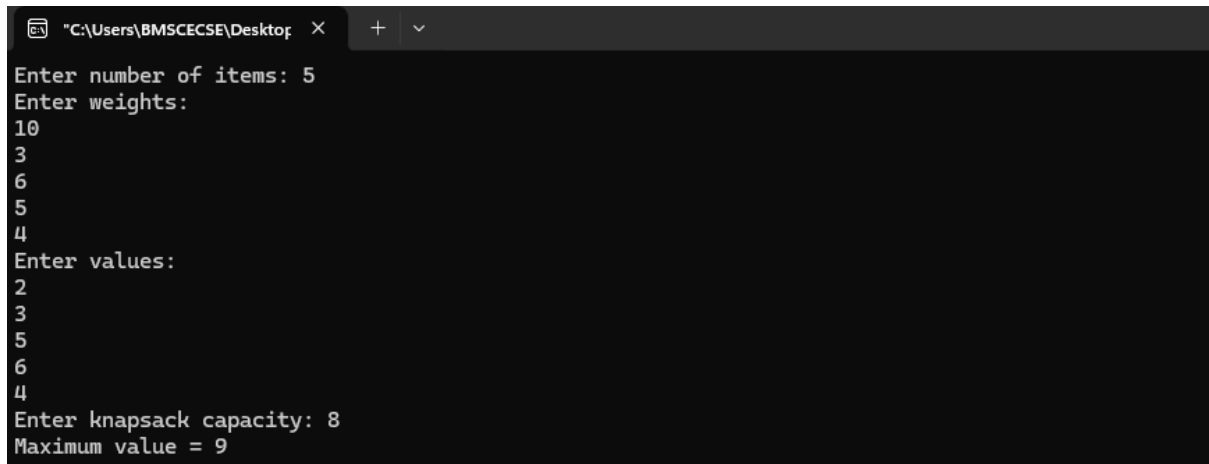
    S
    printf("Enter weights:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &weights[i]);
    }
    printf("Enter values:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &values[i]);
    }
    printf("Enter knapsack capacity: ");
    scanf("%d", &W);
}
```

```
int result = knapsack(W, weights, values, n);

printf("Maximum value = %d\n", result);

return 0;
}
```

Output



```
"C:\Users\BMSCECSE\Desktop" x + v
Enter number of items: 5
Enter weights:
10
3
6
5
4
Enter values:
2
3
5
6
4
Enter knapsack capacity: 8
Maximum value = 9
```

Leet Code: Partition Equal Subset Sum

```
#include <stdbool.h>
```

```
bool canPartition(int* nums, int numsSize) {
    int sum = 0;
```

```
    for (int i = 0; i < numsSize; i++) {
        sum += nums[i];
    }
```

```
    if (sum % 2 != 0) {
        return false;
    }
```

```
    int target = sum / 2;
```

```
    bool dp[target + 1];
```

```
    for (int i = 0; i <= target; i++) {
        dp[i] = false;
```

```
    }  
    dp[0] = true;  
  
    for (int i = 0; i < numsSize; i++) {  
        for (int j = target; j >= nums[i]; j--) {  
            dp[j] = dp[j] || dp[j - nums[i]];  
        }  
    }  
  
    return dp[target];  
}
```

✓ Testcase | > Test Result

Accepted Runtime: 0 ms

✓ Case 1 ✓ Case 2

Input

nums =
[1,5,11,5]

Output

true

Expected

true

✓ Testcase | > Test Result

Accepted Runtime: 0 ms

✓ Case 1 ✓ Case 2

Input

nums =
[1,2,3,5]

Output

false

Expected

false